

Lab 1

In this lab, you'll begin an implementation of one of our first data structures, an array-based list.

Implementing an Array-based List

If you haven't already, please install VS Code on your computer (Google "vs code" to find it).

Once it's installed, click on its icon to start it. In your file system (Finder or File Explorer depending on your OS), create a directory/folder for your work in this class and then a subdirectory/subfolder for this lab. Then, in VS Code, create a new file as a Java file. Name its class `OurArrayList` (making the file `OurArrayList.java`). You'll need to select Save in the File menu to set this name and save the code in the right place.

Now let's begin implementing the class. The "big picture" idea is that objects of this class will store a collection of strings (i.e. objects of type `String`), just like a list sequence of things in the physical world. The class will use an array for the underlying storage. Begin by adding attributes (variables) to the `OurArrayList` class. It should have an array of strings called `vals` and an integer `num`. The latter will store the number of values in the list, which is different than the length of the `vals` array; in general, the array will have empty cells for values that haven't been added yet.

Your first method should be a constructor that takes no arguments. (Recall that a constructor is a method without a return type and whose name is the same as the name of its class. It is called whenever an object of that class is created.) The constructor should set `num` to 0 (since the list is empty) and create an array for `vals`; begin by creating one of length 3.

Next write a method `size` that returns the number of values in the list (i.e. the `num` attribute).

The first non-trivial method is `add`, which should take a string and add it to the array. It should store the string in the first empty array cell (use `num` to determine which one this is), and increment `num` (since the list now has one more value). For now, assume that the array has an empty cell.

In order to test that your code works, you'll need to create a class with the `main` method to test it. Select New File in the File menu and create a new class called `Main`. Again, use Save to put this into the correct directory. Give it a method `public static void main`; recall that this is what is run when you start the program. Inside `main`, create an instance of `OurArrayList` (use `new` to create the object and store a reference to it in a variable of type `OurArrayList`). Then print this object's size, add a value or two, and print its size again. Run your program to check that `add` and `size` seem to work. (To run the program, hit the button with the triangle near the top right.)

Next write a method `contains` that takes a `String` and returns a boolean indicating whether that string is in the list. Use the `equals` method of `String` to see if two strings are equal (e.g. `s1.equals(s2)` will return whether `s1` and `s2` are equal). We'll talk about why you don't want to use `==` later in the class.

Once `contains` is written, modify `main` to test that `contains` returns `false` before a value is added to the list and `true` afterwards.

Next, it's time to revisit `add`. Our first implementation makes the unfortunate assumption that the array always has room for another string. Since we'd like to allow our list to grow bigger than 3 elements, we need to remove this assumption. To do this, `add` should check if the array is already full before putting a new value into it. (To see if it's full, compare `num` to `vals.length`.) If the array is full, create an array that is twice the current length and copy the values from the old array into the new one. Then assign this new array to `vals`; this copies the reference to the new array into the attribute, effectively replacing the old array with the new one. Check that this method works by changing `main` to insert more strings.

After all that, you've written a basic list implementation. If you have additional time, try to implement the

methods below (or other useful list methods). As you implement each method, be sure to modify `main` to test it.

- A method `clear` that takes no arguments and returns nothing, but empties the list. (Hint: you can do this without touching the `vals` attribute.)
- A method `remove` that takes a `String` and removes that value if it appears in the list. If the list contains more than one copy of the `String`, only one of them should be removed.
- A method `addAll` that takes another `OurArrayList` object and adds all members of the argument list.